

Canonical Entity IDs for Grounded Action Generation

A Reference Layer for Small Specialist Models over Structured State

J. D. Bohrman

Independent Researcher

May 28, 2026

Abstract

Small specialist models can produce accurate structured actions when the action schema is narrow and the state is explicit, but they still fail when natural-language instructions refer to entities indirectly. Phrases such as “the best successor”, “the highest-confidence track”, or “the strongest available recon unit” require the model to ground language to one entity among many plausible candidates. We study this problem in a multi-robot coordination setting and argue that the missing abstraction is not merely better prompting but a stable reference layer.

We introduce *Canonical Entity IDs* (CEIDs), a model-facing alias scheme that rewrites domain entities into compact, typed, scope-aware identifiers while preserving a reversible map back to canonical runtime IDs. CEIDs are paired with structured state views, typed action targets, and lightweight ranking hints over candidate entities. In our harder robotics candidate-pressure slice, a baseline specialist without this reference discipline achieved only 61.6% exact-match accuracy on a 250-example evaluation. After adding canonical entity IDs, consistent alias rewriting, and ranked grounding hints, the same specialist reached 98.0% exact-match accuracy, 100% exact operation accuracy, 100% valid JSON, and 99.2% schema validity on the same evaluation slice. The remaining failures are concentrated in a small number of assignment and reassignment cases rather than broad grounding collapse.

These results suggest that reference design is a first-class systems problem for applied AI. Canonical entity IDs provide a simple, inspectable mechanism for improving grounded structured generation without requiring larger base models, broader prompts, or unconstrained agentic reasoning.

1 Introduction

Language models are increasingly asked to issue actions over structured systems rather than write open-ended prose. In such settings, the core problem is often not syntax. It is grounding. A model may know that the requested action is `reassign_task` or `set_lock_state`, yet still choose the wrong robot or track because several candidates are semantically plausible.

This failure mode is common in operational domains where the user rarely names the exact identifier they want. Instead, instructions refer to entities by role, quality, or relational status: the best successor, the highest-confidence contact, the strongest available inspection unit. The structured action problem is therefore also a reference problem.

Code availability. Repository, paper source, and experimental artifacts are available at <https://github.com/comradelemoncake/sst>.

This paper studies a narrow claim: *small specialist models become materially more reliable when the system gives them a consistent, compact, reversible identity layer over structured state*. We call this layer *Canonical Entity IDs* (CEIDs). CEIDs do not replace structured state or typed actions. They sit between the runtime world and the model, converting entities such as robots, tasks, and tracks into compact model-facing aliases while preserving exact reversibility back to the canonical runtime IDs.

The result is a more legible grounding problem. Instead of forcing the model to reason directly over long heterogeneous identifiers, CEIDs give it a typed, scope-aware vocabulary such as `robot.recon.w11.a` or `track.vehicle.w11.2`. Combined with ranking hints over candidate entities, this creates an explicit reference layer for state-to-action generation.

CEIDs also fit naturally into broader specialist-routing systems in which a general controller or interface layer routes user requests to narrow domain models. In such architectures, the routing layer decides *which* specialist to invoke, while CEIDs help the invoked specialist refer to the current entities consistently once it receives structured state.

Contributions.

1. We define Canonical Entity IDs as a reversible alias layer for structured state-to-action generation.
2. We show how CEIDs can be integrated into a validator-backed specialist workflow without changing the runtime action schema.
3. We present a robotics coordination benchmark in which indirect entity references create candidate-pressure grounding failures.
4. We show that CEIDs plus consistent alias rewriting and ranked grounding hints improve exact-match accuracy from 61.6% to 98.0% on a 250-example evaluation slice.
5. We analyze the main implementation failure encountered during development: inconsistent rewriting between alias-bearing state and helper hints, which temporarily caused a severe schema-validity regression.

2 Problem Setting

2.1 Structured Action Generation

We study tasks in which a model must map:

1. a natural-language instruction,
2. a structured state view,
3. into one typed action object.

The specialist is not asked to emit prose. It is asked to produce one executable edit such as task assignment, task reassignment, robot mode change, or track lock-state update.

2.2 Why Grounding Fails

In easy slices, direct references dominate and exact match is straightforward. In harder slices, the instruction deliberately underspecifies the entity and requires model-side resolution from structured state. For example:

- “Assign this task to the strongest available recon unit.”
- “Hand off this task to the best available successor.”
- “Promote the highest-confidence vehicle contact to confirmed.”

Without a disciplined reference layer, the model must simultaneously parse the instruction, rank the candidates, and emit the final runtime IDs. In practice this produced the dominant errors in our harder slice.

3 Canonical Entity IDs

3.1 Definition

A Canonical Entity ID is a model-facing alias with four components:

`<type>.<class>.<scope>.<slot>`

Examples include:

- `robot.recon.w11.a`
- `robot.inspection.w11.c`
- `task.track.w11.2`
- `track.vehicle.w11.5`

Each alias is generated from canonical runtime state and stored with a reversible map. The model sees the aliases; the runtime executes on the original IDs.

3.2 Design Goals

The identifier layer is designed to satisfy five properties:

1. **Typed:** entity kind remains explicit.
2. **Compact:** IDs are shorter and more regular than raw runtime identifiers.
3. **Scoped:** aliases encode world-local scope rather than pretending to be globally meaningful.
4. **Reversible:** every alias has a deterministic reverse mapping to its runtime ID.
5. **Model-friendly:** token patterns are regular and semantically grouped.

3.3 Reference Consistency

CEIDs are only useful if every model-facing reference path is rewritten consistently. That includes:

- candidate ID lists,
- candidate entity objects,
- target entity objects,
- helper summaries and ranking hints,

Listing 1: Representative coordination edit target.

```
{
  "op": "assign_task",
  "taskId": "task.track.w11.2",
  "robotId": "robot.recon.w11.a"
}
```

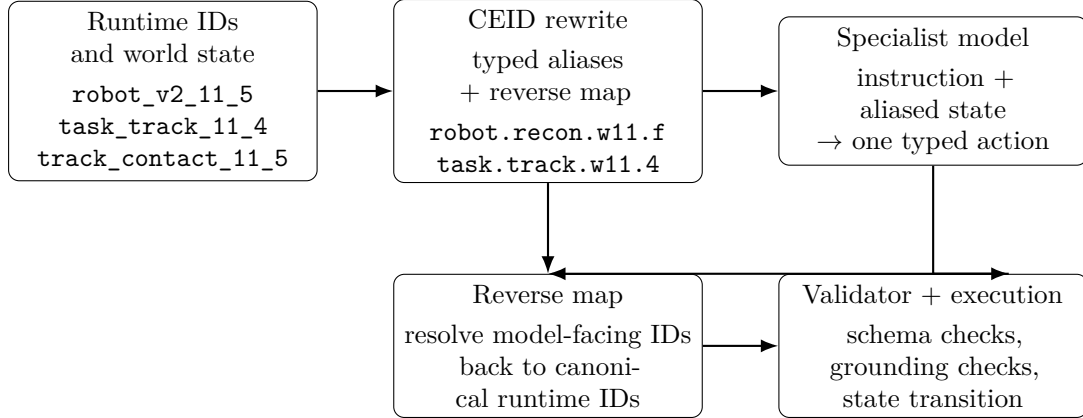


Figure 1: Canonical Entity ID pipeline. Runtime entities are rewritten into model-facing aliases before decoding, then resolved back to canonical IDs before validation and execution. The key requirement is consistency: state views, helper hints, and generated actions must all refer to entities in the same alias space.

- the instruction itself when it contains direct identifiers.

We found this consistency requirement to be operationally important. A temporary bug that left ranking hints in canonical runtime IDs while the rest of the state used CEIDs caused a sharp schema-validity drop despite improving exact-match accuracy. Once the hints were rewritten into the same alias space, performance recovered.

4 System Overview

4.1 Interface Contract

The full runtime action schema remains unchanged. CEIDs are a model-facing translation layer, not a new execution layer. The specialist still emits typed actions such as:

After generation, the action is resolved back to canonical IDs and checked by a schema/domain validator before execution.

4.2 Model-Facing State

Listing 2 shows a representative robotics state view. The important addition is the `selectionHints` block, which contains a small ranked list of likely candidates in the same alias space as the rest of the prompt.

Listing 2: Representative robotics state view with canonical entity IDs and ranking hints.

```
{
  "editKind": "set_lock_state",
  "worldMeta": {
    "id": "world_robotics_11",
    "environment": "urban",
    "clockMs": 1716660011000
  },
  "candidateTrackIds": [
    "track.vehicle.w11.1",
    "track.vehicle.w11.2",
    "track.vehicle.w11.3"
  ],
  "candidateTracks": [
    {
      "id": "track.vehicle.w11.2",
      "classification": "vehicle",
      "confidence": 0.91,
      "lockState": "tracking"
    }
  ],
  "selectionHints": {
    "trackSelection": {
      "recommendedTrackIds": [
        "track.vehicle.w11.2",
        "track.vehicle.w11.1",
        "track.unknown.w11.1"
      ]
    }
  }
}
```

4.3 Validation

Validation remains central. Each generated action is checked for:

1. JSON parseability,
2. schema validity,
3. existence of referenced entities in the current state,
4. basic domain invariants.

This makes CEIDs compatible with the same validator-backed workflow used by structured specialist systems more broadly. The contribution here is not constrained decoding alone, but a cleaner reference interface for the model [1, 6–8].

Table 1: Robotics hard_v2 cumulative ablation on 250 held-out examples. Each row adds one change on top of the previous row.

Variant	Valid JSON	Valid Schema	Exact Op	Exact Match
Baseline specialist	1.000	0.992	1.000	0.616
Baseline + CEIDs and ranked hints	1.000	0.856	1.000	0.660
Prior row + consistent hint rewriting	1.000	0.980	1.000	0.968
Prior row + parser and action normalization	1.000	0.992	1.000	0.980

5 Experimental Setup

5.1 Domain

We evaluate CEIDs in a synthetic but source-grounded multi-robot coordination domain derived from ROS 2, MAVLink/PX4, and peer-synchronization-inspired abstractions [2–5]. The world model contains robots, tasks, tracks, links, and coordination policies. The action space includes:

1. `assign_task`,
2. `reassign_task`,
3. `set_mode`,
4. `set_lock_state`.

5.2 Harder Candidate-Pressure Slice

The paper focuses on the harder `hard_v2` slice rather than the earlier frozen baseline because the hard slice exposes the grounding problem directly. Candidate choice depends on battery, link quality, role fit, GPS degradation, and task constraints. Instructions often refer to entities indirectly rather than by exact runtime name.

5.3 Training Setup

We fine-tune a Qwen2.5-Coder-1.5B base model with LoRA over structured instruction-state-action triples. The main comparison in this draft is between:

1. a baseline structured specialist without the final CEID stack,
2. the same specialist with CEIDs, consistent alias rewriting, and ranked grounding hints.

The current experiments emphasize exact structured action generation rather than broad language quality.

6 Results

6.1 Main Quantitative Result

Table 1 summarizes the current 250-example evaluation slice as a cumulative ablation over the identity-layer stack. Each row adds one change on top of the previous row.

The progression matters more than any single row. The baseline specialist already had perfect operation selection, but failed on entity grounding. Adding CEIDs and ranked hints improved exact

match immediately, but exposed a consistency bug because the helper hints were still written in canonical runtime IDs rather than CEIDs. Once those hints were rewritten into the same alias space as the rest of the prompt, exact match jumped from 66.0% to 96.8%. A final parser and action normalizer cleaned up the remaining generation drift, yielding 98.0% exact match with 99.2% schema validity.

6.2 Failure Evolution

The development path is informative:

1. The baseline specialist selected the right operation but often the wrong entity.
2. Adding ranked hints improved exact match but briefly damaged schema validity because the hints were not rewritten into CEIDs.
3. Fixing hint rewriting restored consistency and produced a large jump in exact match.
4. A final parser and action normalizer cleaned up residual generation drift such as appended garbage tokens and slight field-shape errors.

6.3 What the Remaining Failures Mean

At 98.0% exact match on this slice, the remaining errors are small and concentrated. They no longer reflect broad grounding collapse. Instead they are rare assignment/reassignment misses or residual generation artifacts. This is a materially different regime from the starting point.

7 Discussion

7.1 Why CEIDs Help

The simplest explanation is that CEIDs reduce the entropy of reference resolution:

1. aliases are shorter and more uniform than runtime IDs,
2. entity type is always explicit,
3. scope is local and regular,
4. ranked hints share the same identifier space,
5. the reverse map allows clean execution without exposing runtime complexity to the model.

This does not mean CEIDs are a universal substitute for better modeling. It means that in structured action settings, naming itself is an interface decision with measurable consequences.

7.2 Relation to Broader Specialist Systems

CEIDs fit naturally into a broader specialist architecture in which small models operate over canonical state and typed actions while larger controllers handle routing or user interaction. But the evidence in this draft most directly supports the identity-layer claim, not the entire pluggable-expert story. That narrower contribution is the one this paper makes.

7.3 Relation to Murmuration

The broader Murmuration idea remains relevant as a system-level framing for multi-specialist coordination, but CEIDs should be understood as a concrete primitive rather than the whole theory. They help multiple components refer to the same entities consistently. That makes them compatible with Murmuration-style architectures without requiring this paper to prove the entire broader thesis. In that sense, Murmuration is a reasonable higher-level framing for systems composed of many narrow specialists, while CEIDs are one concrete reference mechanism that can make such coordination legible and reliable at the action layer.

8 Limitations

1. The current benchmark is synthetic, even though it is structured from source-inspired robotics abstractions.
2. The strongest quantitative result reported here is on a 250-example held-out slice rather than the full test set.
3. We currently emphasize one base model family and one primary domain.
4. The present paper does not yet isolate how much of the gain comes from aliasing alone versus aliasing plus ranked hints.
5. The residual action normalization step is transparent and small, but it is still a post-generation cleanup layer and should be reported as such.

9 Next Experiments

The most useful next experiments are:

1. ablate CEIDs against ranked hints more cleanly,
2. run the same identity layer in a second non-robotics domain,
3. test whether CEIDs continue to help under larger candidate sets,
4. compare against stronger constrained-decoding baselines on the same action targets.

10 Conclusion

Canonical Entity IDs provide a simple, reversible reference layer for grounded structured action generation. In a robotics coordination benchmark where the baseline specialist struggled with indirect entity references, CEIDs plus consistent alias rewriting and lightweight ranking hints raised exact-match accuracy from 61.6% to 98.0% on a 250-example evaluation slice while preserving fully structured outputs. The main lesson is straightforward: for applied AI systems that act over structured state, reference design is not incidental plumbing. It is part of the model interface, and it can determine whether structured specialists ground reliably or fail under candidate pressure. More broadly, CEIDs appear to be a useful primitive for larger coordinated specialist architectures in which multiple models must act over the same evolving world state.

References

- [1] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12293–12314. Association for Computational Linguistics, 2023. URL <https://aclanthology.org/2023.emnlp-main.674/>.
- [2] Iroh. Iroh documentation, 2026. URL <https://docs.iroh.computer/>. Accessed 2026-05-26.
- [3] p2panda. p2panda, 2026. URL <https://p2panda.org/>. Accessed 2026-05-26.
- [4] PX4. Px4 autopilot, 2026. URL <https://px4.io/>. Accessed 2026-05-26.
- [5] ROS 2 Project. Ros 2, 2026. URL <https://github.com/ros2/ros2>. Accessed 2026-05-26.
- [6] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.emnlp-main.779/>.
- [7] Richard Shin, Christopher H. Lin, Sam Thomson, Charles Chen, Subhro Roy, Emmanouil Antonios Platanios, Adam Pauls, Dan Klein, Benjamin Van Durme, and Jason Eisner. Constrained language models yield few-shot semantic parsers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7699–7715. Association for Computational Linguistics, 2021. URL <https://aclanthology.org/2021.emnlp-main.608/>.
- [8] Mengzhou Xia, Yuxiu Jiang, Yao Wu, Zhiwei Tang, Yunpeng Luo, Jimmy Lin, Rui Zhang, and Qun Liu. Generating structured outputs from language models: Benchmark and studies. *arXiv preprint arXiv:2501.10868*, 2025. URL <https://arxiv.org/abs/2501.10868>.